

CS455 Project Proposal

CampusEcho: Anonymous & Accountable Feedback System

Team Members

- **Kishore Senthilkumar** – Roll No: 230566
- **B.Vaishnavi** – Roll No: 230295
- **M.Amara Gayathri** – Roll No: 230624
- **Reddi Pallavi** – Roll No: 230850
- **Poorvie Sadagopan** – Roll No: 230759
- **Vundela Obula Reddy** – Roll No: 231178

1. Project Description

CampusEcho is an accountable, privacy-preserving course/faculty feedback platform. It enables students to submit reviews anonymously while providing the institute with an automated moderation pipeline. The system utilizes an AI gateway to detect hate speech/abuse and spam/coordinated brigading, routing user appeals against automated moderation to a decentralized, pull-based moderation queue.

2. User Roles

- **Student (Submitter):** Submits anonymous feedback, views published ratings, and initiates appeals if their content is flagged and removed by the AI/moderator.
- **Moderator:** Actively pulls appeal tickets from the moderation queue, manually reviews contested feedback, resolves tickets.

3. Core Transactional Workflow

The system governs the stateful lifecycle of feedback: **Submitted** → **AI Gateway (Published/Flagged)** → **Appealed (Ticket Raised)** → **Locked by Moderator** → **Restored/Deleted**. Each transition impacts the visibility of the review.

4. Shared/Contended Resource & Concurrency Challenge

- **The Resource:** A central pool of moderation tickets generated exclusively from user appeals against AI-flagged feedback.
- **The Challenge:** Moderators actively pull from a central queue. When an appeal ticket is raised, multiple moderators will concurrently attempt to claim it. The backend must utilize atomic MongoDB operations to lock the ticket, ensuring it is allocated to exactly one moderator, preventing double-processing and race conditions.

5. Agentic AI Integration

- **Placement & Tools:** An Autonomous Triage Agent operates asynchronously via Go worker pools (goroutines), analyzing incoming feedback streams without blocking user submissions. It utilizes internal APIs to assess course submission metrics and update feedback states.
- **Boundary (Micro Level - Autonomous):** Analyzes individual text to publish normal reviews or flag explicit hate speech/abuse.
- **Boundary (Macro Level - Autonomous):** Monitors submission velocity and semantic similarity to detect spam/coordinated brigading attacks via a rule-based scoring system:
 - **Metrics:** Calculates a Brigading Risk Score using the Velocity Metric (V : reviews for a course per 10-minute window) and Semantic Similarity Metric (S : average cosine similarity of text embeddings across the batch).

- **Thresholds:** A batch is flagged if it breaches both the Spike Threshold ($V > 15$) and Similarity Threshold ($S > 75\%$, applied only to reviews exceeding 5 words to prevent false-flagging generic 1-word submissions).
- **Boundary Cases for Testing:**
 - * *Post-Exam Organic Surge:* $V = 30$, $S = 30\%$. System Decision: **Allow** (organic feedback).
 - * *Slow Drip Spam:* $V < 15$, $S = 95\%$. System Decision: **Micro-Flag only** (duplicate spam).
 - * *Coordinated Attack:* $V = 20$, $S = 85\%$. System Decision: **Macro-Flag triggered**.
- **Boundary (Prohibited):** The agent is strictly barred from accessing identity-mapping tables.

6. Proposed Technology Stack

- **Frontend:** React.js with Tailwind CSS (for rapid scaffolding of Student and Moderator dashboards).
- **Backend:** Go (monolithic architecture utilizing goroutines for concurrent background processing).
- **Database:** MongoDB (utilizing atomic operations for resource locking).
- **AI API:** Claude / AWS Bedrock (for the triage agent's decision logic).

7. Initial Project Risks

1. Guaranteeing absolute concurrency correctness across distributed Go microservices under heavy load.
2. LLM hallucinations causing the agent to misclassify legitimate critical feedback as spam/brigading.
3. Maintaining complete decoupling of user authentication from the feedback payload.

8. Project Management & Links

The project will follow a strict sprint cycle managed in Jira. Every requirement will map to a Jira issue, which will dictate a GitHub branch and PR, ensuring end-to-end traceability.

- **Jira Project Link:** <https://kishore200519.atlassian.net/jira/software/projects/SCRUM/summary>
- **GitHub Repository Link:** <https://github.com/Kishore200524/CS455-Project>